JPA

Hasta ahora, con el driver J, escribíamos a mano el código SQL (igual que lo haríamos en la consola) y recorremos los ResultSet que obtenemos como resultado de nuestras querys. El proceso es muy similar al que hacemos manualmente al operar con la base de datos, aunque sea desde nuestro programa Java

Con los ORM (Objetc-Relational Mapping o Herramientas de Mapeo Objeto-Relacional) creamos clases que se identifican con las tablas de nuestra base de datos y nos olvidamos casi por completo del SQL. Se realiza una identificación biunívoca entre Tablas SQL y Objetos Java. Si nos paramos a pensarlo un poco la semejanza es inmediata: una tabla es una clase y cada fila de la tabla, un objeto

JPA (antes Java Persistence API y ahora conocido como Jakarte Persistence) es un ORM que especifica un conjunto de interfaces y anotaciones para trabajar con bases de datos (relacionales, fundamentalmente) pero por si misma no sirve para trabajar. Existen numerosas implementaciones de JPA de las que algunas de las mas usadas son las siguientes:

- Hibernate. Sin duda la mas usada de todas con diferencia. Es la que usa por defecto Spring Boot
- EclipseLink. Es la implementación de Oracle (propietario de Java, de Eclipse y de la propia JPA) Incluye extensiones para bases de datos noSQL como MongoDB
- Apache OpenJPA. Hoy en desuso casi por completo
- DataNucleus. Interesante porque es quizás la mas usada con bases de datos noSQL (MongoDB, Cassandra, Hbase, Neo4j...). También funciona, por supuesto, con bases de datos SQL
- ObjectDB. Importante porque incluye su propio motor de bases de datos orientado a objetos

Hay que tener en cuenta que JPA se diseño pensando en bases de datos relaciones. NoSQL apenas tenía uso en el momento de su diseño y por tanto no está realmente pensado para estas arquitecturas y su uso actual presenta muchas limitaciones.

CREANDO UN PROYECTO EN ECLIPSE CON HIBERNATE

Creamos un proyecto Maven. Marcamos en la opción de **"Create a simple project"**
En la segunda página completamos los siguientes campos:
group id: com.dam2.goya
artifact id: adat-jpa
Nos aseguramos de que **packaging** este configurado como **jar**

NOTA: tanto group id como artifact id actúan como identificadores únicos del proyecto. Se los suele llamar también coordenadas
- **group id** debería de identificar a la empresa que lo publica y tiene el formato de una url al revés
- **artifact id** sería el nombre del proyecto

Una vez creado el proyecto abrimos el archivo pom.xml y copiamos lo siguiente:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
	https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<groupId>com.dam2.goya</groupId>
	<artifactId>adat-jpa</artifactId>
	<version>0.0.1-SNAPSHOT</version>

	<properties>
		<maven.compiler.release>17</maven.compiler.release>

	<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
	</properties>

	<dependencies>
		<dependency>
			<groupId>org.slf4j</groupId>
			<artifactId>slf4j-nop</artifactId>
			<version>2.0.16</version>
		</dependency>

		<!-- Hibernate (incluye la API de Jakarta Persistence) -->
		<dependency>
			<groupId>org.hibernate.orm</groupId>
			<artifactId>hibernate-core</artifactId>
			<version>7.4.2.Final</version>
		</dependency>

		<!-- Driver J de MySQL (el mismo que usábamos con JDBC) -->
		<dependency>
			<groupId>com.mysql</groupId>
			<artifactId>mysql-connector-j</artifactId>
			<version>9.1.0</version>
		</dependency>
	</dependencies>
</project>
```

Si vamos a usar MariaDB en lugar de MySQL es conveniente cambiar el driver por este:

```
<dependency>
	<groupId>org.mariadb.jdbc</groupId>
	<artifactId>mariadb-java-client</artifactId>
	<version>3.5.1</version>
</dependency>
```

Importante: tienes que personalizar algunos campos según los valores de tu proyecto. Observa que al principio aparece el groupID y el artifactID del proyecto Maven

POM significa Project Object Model y viene a ser como el fichero de configuración principal de Maven. Luego veremos que JPA/hibernate tienen su propio fichero de configuración que se llama persistence.xml

Y, ya que estamos, un proyecto Maven es un proyecto Java que sigue unas convenciones concretas de gestión y construcción definidas por Apache (de hecho, se lo suele llamar también Apache Maven). Maven no son siglas de nada. Es una palabra que viene del idioma judío y que significa “experto” o “acumulador de conocimiento”

Una vez escrito el pom.xml vamos al menú de Project y pulsamos en Update Maven Project. Deberíamos de hacerlo siempre que hagamos cambios en este fichero.

Ahora creamos una nueva carpeta (OJO: Folder, no Source Folder) que se llame META-INF dentro de la carpeta src/main/resources. Dentro de ella creamos un archivo que se llame persistence.xml e incluimos en él lo siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
             https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd"
             version="3.2">

    <persistence-unit name="clasePU">
        <class>com.dam2.goya.Producto</class>

        <properties>
            <property name="jakarta.persistence.jdbc.url"
                  value="jdbc:mysql://localhost:3306/clase_jpa?
                  createDatabaseIfNotExist=true"/>
            <property name="jakarta.persistence.jdbc.user" value="profe"/>
            <property name="jakarta.persistence.jdbc.password" value="abc123"/>

            <!-- Crea/actualiza las tablas a partir de las entidades -->
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <!-- Muestra el SQL que genera Hibernate -->
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

Con MariaDB cambiar la cadena de conexión por esta:

```
      <property name="jakarta.persistence.jdbc.url"
            value="jdbc:mariadb://localhost:3306/clase_jpa?
            createDatabaseIfNotExist=true"/>
```

Como dijimos antes, este es el archivo de configuración de jpa/hibernate. También aquí tendríamos que personalizar y añadir algunas cosas. Tendremos que poner una linea class por cada entidad nueva que use nuestra app. En este ejemplo usaremos únicamente una que se llama Productos. A la entidad se le antepone el nombre del group id definido. Esta entidad se corresponderá con una tabla de nuestra base de datos.
También tenemos que personalizar, al menos, los campos de usuario y contraseña. Puede que también la cadena de conexión

Ahora creamos dentro de src/main/java un package que se llame como nuestro group id (com.dam2.goya) y dentro del mismo una clase que se llame como la entidad (Producto). Copiamos dentro este contenido:

```
package com.dam2.goya;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
//@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // @Column(name = "producto", length = 100, nullable = false, unique = true)
    private String nombre;

    // @Column(nullable = false)
    private double precio;

    // JPA exige siempre un constructor sin argumentos
    public Producto() {
    }

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }
```

```
    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Una anotación en Java es un metadato de información adicional que no forma parte de la lógica del programa ni se compila pero que le da información a algún elemento del mismo. Ya vimos algunas el curso pasado (@Override, por ejemplo, recuerda). En este caso las anotaciones que incluimos le sirven a JPA e Hibernate para tener mas información sobre lo que tienen que hacer con la base de datos. En el anterior ejemplo hay algunas anotaciones que son obligatorias. Otras están comentadas para que veas como afectan al proyecto al descomentarlas.

@Entity es obligatoria. Indica que esta clase es una Entidad de JPA/Hibernate
@Table permite que el nombre de la tabla no tenga que coincidir con el de la entidad
@Id indica la clave primaria. En hibernate es obligatorio que toda tabla tenga una clave primaria.
@GeneratedValue tal y como aquí aparece es el equivalente de un AUTO INCREMENT
@Column da información adicional a la hora de crear ese campo en la tabla

Cada atributo de la clase se convertirá en una columna de la tabla. Las anotaciones solo son necesarias en las columnas en las que queremos introducir alguna información o modificación sobre el comportamiento base. Por ejemplo, en un campo String si no especificamos el tamaño tomará por defecto 255 caracteres.

Vamos a hacer ahora un programa que use esta entidad. Nuestro programa creará la base de datos y la tabla si no existen y luego insertarán un registro

```
package com.dam2.goya;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

import java.util.logging.Level;
import java.util.logging.Logger;

public class Main {

      // Atributo estático para deshabilitar el logger de hibernate
     //private static final Logger LOG_HIBERNATE =
           Logger.getLogger("org.hibernate");
```

```
    public static void main(String[] args) {
      // Le decimos a hibernate que solo muestre los errores y no
      mensajes informativos
      //LOG_HIBERNATE.setLevel(Level.SEVERE);

         EntityManagerFactory emf =
                 Persistence.createEntityManagerFactory("clasePU");
         EntityManager em = emf.createEntityManager();
         try {
            em.getTransaction().begin();
            Producto p = new Producto("Teclado", 25.5);
            em.persist(p);
            em.getTransaction().commit();
            System.out.println("Producto guardado con id " + p.getId());
         }catch(Exception e) {
            System.out.println("Error: " + e.getMessage());
            em.getTransaction().rollback();
         }
         em.close();
         emf.close();
    }
}
```

El nombre de clasePU que se le da a la entidad principal puede ser cualquiera que prefieras. Se suele poner PU como final por convención. PU son las siglas de Persistence Unit.

Las únicas lineas fuera del andamiaje necesario para operar con hibernate son aquellas en las que se crea el producto y se graba en la base de datos (con el método persist()). Hibernate automaticamente insertará una línea en la tabla correspondiente. La base de datos y la tabla se crearan automáticamente si no existen. De hecho, aunque no insertemos ningún producto (prueba a comentar las líneas 23, 24 y 27 de este último código) la base de datos y la tabla se crearan igualmente si no existen. La necesidad de hacer commit() es la opción por defecto, al revés que ocurría con el driver J.

Habrás observado que la salida a consola de hibernate es verborreante y aparece en rojo porque usa System.err en lugar de System.out. Para hacerla desaparecer descomenta en el main las dos líneas referentes al Logger. Aún quedarán las relativas a SLF4J (Simple Logging Facade for Java: una fachada de logging simple para Java). Esos debemos de deshabilitarlos en el fichero POM.xml borrando o comentando las líneas correspondientes a esta dependencia.

También verás que aparecen las sentencias SQL que se han usado: create table e insert. No aparece el create database, pero se está haciendo también. Si no queremos que estas líneas aparezcan en consola tenemos que poner a false el parámetro hibernate.show_sql en el fichero persistence.xml

Veamos las piezas de código que necesitaríamos para hacer un CRUD:
Para insertar un registro ya lo hemos visto antes:

```
Producto p = new Producto("Teclado", 25.5);
em.persist(p);
```

Para modificar un producto (update). En el siguiente ejemplo primero lo buscamos por clave primaria y luego lo modificamos:

```
Producto p = em.find(Producto.class, 1L);
if (p != null)
     p.setPrecio(29.9);
```

Para eliminar un producto, igualmente primero lo buscamos:

```
Producto p = em.find(Producto.class, 1L);
if (p != null)
     em.remove(p);
```

Después de cualquiera de estos cambios hay que hacer commit, no lo olvides, pero fíjate que sólo hace falta ejecutar persist() con los objetos nuevos. Los objetos ya existentes que encontramos con find() están ya "gestionados" por hibernate y cuando hacemos commit se transmiten los cambios a las tablas, ya sean updates o deletes.